

DealFlow360

First Two Hero Features

Feature Explanation → Algorithm → Example → Edge Cases → Recommended Optimization

This document focuses only on the first two DealFlow360 hero intelligence features. It first explains Feature 1 completely and then Feature 2, so the reader can understand the business problem before seeing the algorithmic implementation. The original project requirements are retained, while bounded LP/MILP and bitmask change vectorization are added as proposed engineering enhancements.

1. What These Two Hero Features Actually Do

DealFlow360 is intended to behave as a controlled deal decision engine rather than simply storing quotations. The first hero feature controls the commercial offer before approval. The second controls what happens when the customer negotiates after approval.

Feature	Simple Meaning	Core Decision
Feature 1 — Intelligent Discount & Approval	Check whether the price/discount is safe and decide who must approve it.	Should this offer be approved, and why?
Feature 2 — Customer Negotiation → Automatic Re-approval	Check whether customer changes make the old approval invalid.	Did the negotiated change invalidate the previous approval?

2. Feature 1 — Intelligent Discount & Approval Engine

2.1 What is the feature?

When a Sales Rep creates a quotation, the system should not simply calculate the total and wait for someone to manually decide what to do. It should inspect every quotation line against the commercial policies configured by the Admin, calculate the financial/risk exposure, and automatically determine whether the quotation can proceed or which authorized approver is required.

The original specification defines customer/product/category/contract/campaign discount limits, line-level checks, blended exposure, explainable weighted risk, approval thresholds and dynamic approval routing.

2.2 Why this feature is needed

- A customer may have one discount ceiling while a product category has a stricter ceiling.
- A quote can contain many individually small exceptions that become a large overall concession.
- High discount does not only affect price; it can reduce margin and increase commercial exposure.
- Manual approval creation is slow and can be inconsistent.
- Managers need to know exactly why a quotation was routed for approval.

2.3 Example from the project

Item	Value	Discount	Applicable Limit	Decision
Laptop	₹10,00,000	12%	15% hardware	Pass
Setup Service	₹50,000	18%	10% service	Violation
Support	₹30,000	5%	10% service	Pass

Assume the customer is Gold and has a general 15% ceiling. The Setup Service still fails because the service category allows only 10%. Therefore, the system must evaluate the applicable line/category rule instead of checking only the customer's overall percentage.

2.4 Effective-limit algorithm

For each quote line, calculate the strictest applicable ceiling:

```
effective_limit = MIN(customer_limit,
                      product_limit,
                      category_limit,
                      contract_limit,
                      campaign_limit,
                      other_applicable_limits)
```

This is a hard-policy calculation. It should not be replaced by ML or by an optimization objective because a hard business constraint must remain enforceable and auditable.

2.5 Important anti-bypass check

A Sales Rep should not be able to avoid a discount rule simply by lowering the unit selling price instead of entering a discount. The engine should calculate the effective discount against the applicable reference/list price.

```
effective_discount = 1 - (selling_price / reference_price)
```

This protects the policy engine from a common pricing-bypass case.

2.6 Risk scoring

After policy evaluation, DealFlow360 converts the commercial exposure into an explainable risk score. The source specification uses configurable factors such as discount excess, margin breach, deal value, multiple violations and historical behavior.

Risk Factor	Illustrative Maximum
Discount excess	40
Margin breach	40
Deal value	15
Multiple violations	10
Historical behavior	15

Illustrative classification: 0–20 = Low, 21–40 = Medium, 41–70 = High, 71–100 = Critical. These values should be configurable and versioned as business policy.

```
quote_risk = discount_risk + margin_risk + deal_value_risk
            + violation_risk + other_configured_factors
```

2.7 Blended discount

Line-level checking alone is not enough. The system should also calculate the value-weighted/blended discount so many small exceptions cannot hide a material overall concession.

```
weighted_discount = Σ(line_value × line_discount) / Σ(line_value)
```

For the example, the blended discount is approximately 12.29%. Even though the blended number is below the customer's 15% ceiling, the service line still violates its 10% category limit. This demonstrates why both line-level and quote-level checks are required.

2.8 Approval routing

Risk / Condition	Automatic Route
Low + within policy	Auto-approve

Medium	Sales Manager
High or margin concern	Sales Manager + Finance
Critical / major exception	Senior Manager / Director

The router selects the highest required authorized approval level. The requester must never approve their own request.

2.9 Our recommended optimization: bounded LP / MILP

The original document specifies rule-based discount governance and weighted risk scoring. We recommend adding bounded linear optimization as an additional layer—not as a replacement for the rules.

Layer	Purpose	Recommended Technique
1. Policy	Determine what is allowed	Deterministic rules
2. Optimization	Find the best feasible commercial concession	Bounded LP
3. Discrete decisions	Handle yes/no or integer choices	MILP when required
4. Risk	Explain how risky the result is	Weighted scoring
5. Routing	Determine who approves	Threshold + rule routing

Example: if the customer asks for 18% but the optimizer finds that 12.7% is the highest discount that preserves all hard constraints, the system can generate a safe counter-offer of 12.7%. The optimizer is allowed to search only inside the feasible region.

```

Variables: d_i = discount for line i
Bounds:    0 <= d_i <= effective_limit_i
Constraint: margin >= minimum_margin
Constraint: total concession <= exposure_limit
Objective: maximize commercially useful concession subject to all hard constraints

```

Use MILP rather than plain LP when the decision includes binary/integer choices, such as whether an exception is selected or whether a discrete commercial option is activated.

2.10 Feature 1 — Complete flow

```

Sales Rep creates quote
↓
Validate discount / price inputs
↓
Calculate effective limit for every line
↓
Check hard constraints + anti-bypass pricing
↓
Optional bounded LP/MILP → best feasible concession
↓
Calculate blended exposure + margin impact
↓
Explainable risk score
↓
Dynamic approval routing
↓
Approved / Rejected / Counter-offer / Escalated

```

2.11 Feature 1 — Edge cases and solutions

Edge Case	Solution
Discount >100%	Reject before risk calculation.
Negative discount	Reject unless surcharge behavior is explicitly modeled.
Missing product cost	Do not assume zero cost; mark margin unknown and require review if margin is a gate.

Price lowered instead of discount	Calculate effective discount from reference/list price.
One risky line hidden inside many safe lines	Perform line-level checks plus blended quote exposure.
Different product/category limits	Use the strictest applicable limit for each line.
Quote splitting to avoid approval	Where policy permits, aggregate related quotes by customer/opportunity/time window.
Customer tier changes	Re-evaluate current draft using current policy; preserve historical evaluation inputs.
Cost changes after approval	Recalculate; create a new version if the economic change is material.
Requester is also approver	Block self-approval and route upward.
Approval expires	Expire and re-evaluate.
Currency changes	Use an explicit FX snapshot and re-evaluate.
Bundle manipulation	Validate bundle economics and component restrictions.
Hard margin floor vs promotion	Soft campaign exceptions cannot override hard safety constraints.

2.12 What the manager should see

The UI should show a decision card such as:

RISK: HIGH

Reason:

- Setup Service discount = 18%, allowed = 10%
- Margin exposure detected
- Quote contains multiple commercial exceptions

Recommended route: Sales Manager + Finance

Optional optimized counter-offer: 12.7%

3. Feature 2 — Customer Negotiation → Automatic Re-approval

3.1 What is the feature?

After a quotation is approved, the customer can negotiate. The dangerous implementation would be to edit the approved quote directly. DealFlow360 instead treats negotiation as a new proposal/version. The original approved version remains immutable. The system compares the new version with the last approved version, determines whether the change is material, recalculates risk and automatically sends the new version back through approval when required.

3.2 Why this feature is needed

- An approved quote represents a specific commercial decision at a specific point in time.
- Changing discount, quantity or payment terms can materially change risk.
- A previous approval must not silently carry forward to materially different terms.
- Managers need to see exactly what the customer changed.
- The customer should only be able to finalize the latest valid approved version.

3.3 Project example

Version	Change	Risk	State
V1	8% discount, 30-day payment	31	Approved
V2	18% discount, 60-day payment	78	Reapproval required
V3	Manager counter-offer: 12%, 30 days	Lower	Awaiting approval
V4	Customer accepts latest approved terms	Policy compliant	Confirmed

The critical rule is that V1 remains unchanged. V2 does not inherit V1's approval simply because it is the same quote record; it is a new commercial version that must be evaluated.

3.4 Fields to compare

Field	Why it matters
Price	Changes commercial value and margin.
Quantity	Changes deal value and inventory exposure.
Discount	Changes concession and margin.
Products	Can introduce new restrictions, cost or stock.
Deal value	Can cross approval thresholds.
Payment terms	Can increase financial/credit exposure.
Subscription terms	Can change recurring value and duration.
Currency	Can change economics and thresholds.
Delivery / fulfillment	Can change shipping feasibility and cost.

3.5 Our recommended optimization: bitmask change vectorization

The source document requires field-level diff and material-change detection. We recommend representing the first-pass change result as a bitmask. This is a compact implementation technique for quickly identifying which categories changed.

Bit	Flag	Numeric Value
0	PRICE_CHANGED	1
1	QUANTITY_CHANGED	2
2	DISCOUNT_CHANGED	4
3	PAYMENT_TERMS_CHANGED	8
4	DELIVERY_CHANGED	16
5	CURRENCY_CHANGED	32
6	PRODUCT_CHANGED	64
7	SUBSCRIPTION_CHANGED	128

```
change_mask = PRICE_CHANGED | QUANTITY_CHANGED | DISCOUNT_CHANGED | PAYMENT_TERMS_CHANGED
              = 1 | 2 | 4 | 8
              = 15
```

A material-field mask can be used for a fast first-pass test: if $(\text{change_mask} \& \text{material_mask}) \neq 0$, at least one material field changed. The bitmask identifies WHAT changed; it does not decide HOW significant the change is.

3.6 Why bitmask alone is not enough

A discount change from 10% → 10.5% and 10% → 30% both set DISCOUNT_CHANGED. Therefore the engine must calculate the actual delta after detecting the flag.

```
field comparison
  ↓
bitmask: DISCOUNT_CHANGED
  ↓
actual delta: +0.5 points OR +20 points
  ↓
materiality rules
  ↓
risk recalculation
  ↓
reapproval decision
```

3.7 Material-change detection

A change should be considered material when it affects financial exposure, violates policy, increases risk materially, or crosses an approval threshold.

```
material_change = financial_change
                  OR policy_violation
```

OR risk_increase
OR approval_threshold_crossed

Examples: a large discount increase, a significant quantity increase, longer payment terms, a new product with a different restriction, or a currency change may trigger re-evaluation.

3.8 Risk delta

$\text{risk_delta} = \text{new_risk_score} - \text{previous_approved_risk_score}$

For V1 → V2, risk moves from 31 to 78, so risk_delta = +47. The engine should consider both the absolute new risk and the change from the previously approved state.

3.9 State machine

```
APPROVED
  ↓ customer proposes change
PROPOSED (new version)
  ↓
UNDER_REVIEW
  ├── APPROVED → CURRENT
  ├── REJECTED
  └── COUNTER_OFFER → PROPOSED
```

The state machine prevents invalid transitions such as accepting a rejected version or silently replacing an approved version.

3.10 Feature 2 — Complete flow

```
Latest approved V1
  ↓
Customer negotiates
  ↓
Create immutable V2
  ↓
Compare V1 vs V2
  ↓
Generate bitmask
  ↓
Calculate actual deltas
  ↓
Material-change rules
  ↓
Recalculate economics + risk
  ↓
Risk delta / threshold check
  ↓
State machine
  ↓
Reapproval / Counter-offer / Rejection
  ↓
Only latest valid approved version can be confirmed
```

3.11 Feature 2 — Edge cases and solutions

Edge Case	Solution
Higher discount	Create new version, recalculate risk and route according to new risk.
Lower discount	Recalculate; apply configured auto-review/reapproval policy and retain history.
Quantity increase	Recalculate deal value, margin, stock and approval thresholds.
Quantity decrease	Recalculate because pricing tiers/bundles may change.
Product added	Validate its discount, margin, restrictions and inventory.

Product removed	Validate bundle/service dependencies and recalculate.
Longer payment terms	Treat as financial material change when policy requires; route Finance.
Subscription duration change	Recalculate recurring value, proration and discount.
Delivery change	Recalculate shipping/tax/fulfillment economics if material.
Repeated negotiations	Create V2, V3, V4...; never overwrite history.
Old version accepted	Reject acceptance unless it is the latest valid approved version.
Concurrent edits	Use optimistic version checks and reject stale writes.
Duplicate negotiation request	Use idempotent request/version identifiers.
Negotiation after expiry	Require renewal/reissue before continuing.
Currency change	Create new version, use FX snapshot and re-evaluate.
Customer edits internal cost/margin/risk	Backend rejects protected fields regardless of frontend behavior.
Approval expires during negotiation	Expire the approval and re-evaluate the current version.
Price changes during negotiation	Use version-specific price snapshots and explicit repricing rules.

3.12 What the manager should see

QUOTE V2 – REAPPROVAL REQUIRED

Changes from approved V1:

- Discount: 8% → 18%
- Payment terms: 30 → 60 days

Risk: 31 → 78 (+47)

Status: HIGH – Finance + Sales Manager approval required

Original V1 approval: PRESERVED

4. How Feature 1 and Feature 2 Work Together

The strongest demonstration is not two isolated algorithms. It is one connected decision chain: Feature 1 establishes the approved commercial baseline, and Feature 2 protects that baseline when the customer negotiates.

```

QUOTE CREATED
↓
FEATURE 1: Discount Rules → LP/MILP → Risk → Approval
↓
V1 APPROVED
↓
CUSTOMER NEGOTIATES
↓
FEATURE 2: V2 → Bitmask Diff → Materiality → Risk Delta
↓
REAPPROVAL / COUNTER-OFFER
↓
LATEST VALID APPROVED VERSION
↓
CUSTOMER CONFIRMATION

```

5. Final Recommended Architecture

Hero	Core Algorithm	Our Enhancement	Output
1. Discount & Approval	Rule-based risk + dynamic routing	Bounded LP / MILP for best feasible concession	Explainable risk + approval route + optional counter-offer
2. Negotiation → Reapproval	Version diff + materiality + risk delta + state machine	Bitmask change vectorization	Immutable version + changed-field mask + reapproval decision

6. Why We Are Not Using ML as the Main Engine

The source documents position historical ML prediction as a future enhancement. The current business decisions are primarily policy and workflow decisions: discount ceilings are explicit, approval routes are known, and a negotiated version must be compared with the previous approved version. Therefore deterministic rules, optimization and state machines provide stronger auditability and control for the hackathon implementation.

Question	Best Fit
What is the company allowed to do?	Rules
What is the best feasible commercial option?	LP/MILP
What changed between two quote versions?	Diff + bitmask
Is the change materially important?	Business rules + deltas
Who must approve?	Threshold/rule routing
What is likely to happen in the future?	ML — future enhancement

7. Implementation Notes

- Keep hard constraints separate from optimization objectives.
- Use MILP whenever binary or integer decision variables are required.
- Treat bitmasking as a fast representation of changed fields, not as the materiality decision itself.
- Version every quote and every policy configuration used in an evaluation.
- Keep approval history immutable and auditable.
- Expose the reasons and factor contributions behind every automated decision.
- Enforce customer restrictions at the backend boundary, not only in the UI.
- Only the latest valid approved version can be finalized.

8. Source Basis

Business requirements, examples, edge cases and original algorithm framing are based on the supplied DealFlow360_Overall_Project_Document.pdf, DealFlow360_Detailed_Features_Edge_Cases_Algorithms.pdf and DealFlow360.pdf. Bounded LP/MILP and bitmask flag vectorization are proposed engineering enhancements added to strengthen the original rule-based/versioned architecture.